

C++ cheatsheet

C++ quick reference cheat sheet that provides basic syntax and methods.

Getting Started

hello.cpp

```
#include <iostream>

int main() {
    std::cout << "Hello QuickRef\
    return 0;
}
```

Compiling and running

Variables

```
int number = 5;           // Integer
float f = 0.95;           // Floating
double PI = 3.14159;     // Floating
char yes = 'Y';          // Character
std::string s = "ME";    // String (
bool isRight = true;     // Boolean

// Constants
const float RATE = 0.8;
```

Primitive Data Types

Data Type	Size	Range
int	4 bytes	-2 ³¹ to 2 ³¹ -1
float	4 bytes	N/A
double	8 bytes	N/A
char	1 byte	-128 to 127

Data
Type

Size

Range

User Input

```
int num;

std::cout << "Type a number: ";
std::cin >> num;

std::cout << "You entered " << nu
```

Swap

```
int a = 5, b = 10;
std::swap(a, b);

// Outputs: a=10, b=5
std::cout << "a=" << a << ", b="
```

Comments

```
// A single one line comment in C

/* This is a multiple line comment
in C++ */
```

If statement

```
if (a == 10) {
    // do something
}
```

See: Conditionals

Loops

```
for (int i = 0; i < 10; i++) {
    std::cout << i << "\n";
}
```

See: Loops

Functions

```
#include <iostream>

void hello(); // Declaring

int main() { // main function
    hello(); // Calling
}

void hello() { // Defining
    std::cout << "Hello QuickRef!"
}
```

See: Functions

```
int i = 1;
int& ri = i; // ri is a reference

ri = 2; // i is now changed to 2
std::cout << "i=" << i;

i = 3; // i is now changed to 3
std::cout << "ri=" << ri;
```

ri and i refer to the same memory

```
#include <iostream>
namespace ns1 {int val(){return 5}
int main()
{
    std::cout << ns1::val();
}
```

```
#include <iostream>
namespace ns1 {int val(){return 5}
using namespace ns1;
using namespace std;
int main()
{
    cout << val();
}
```

Namespaces allow global identifiers under

C++ Arrays

Declaration

```
std::array<int, 3> marks; // Defi
marks[0] = 92;
```

Manipulation

92	97	98	99	98	9
----	----	----	----	----	---

Displaying

```
char ref[5] = {'R', 'e', 'f'};
```

```
std::array<int, 3> = {92, 97, 98}

// With empty members
std::array<int, 3> marks = {92, 9
std::cout << marks[2]; // Outputs
```

```
std::array<int, 6> marks = {92, 9

// Print first element
std::cout << marks[0];

// Change 2th element to 99
marks[1] = 99;

// Take input from the user
```

```
// Traditional for loop
for (int i = 0; i < sizeof(ref);
    std::cout << ref[i];
}
```

Multidimensional

	j0	j1	j2	j3	j4	j5
i0	1	2	3	4	5	6
i1	6	5	4	3	2	1

```
int x[2][6] = {
    {1,2,3,4,5,6}, {6,5,4,3,2,1}
};
for (int i = 0; i < 2; ++i) {
    for (int j = 0; j < 6; ++j) {
        std::cout << x[i][j] << "
    }
}
```

C++ Conditionals

If Clause

```
if (a == 10) {
    // do something
}
```

```
int number = 16;

if (number % 2 == 0)
{
    std::cout << "even";
}
else
{
    std::cout << "odd";
}

// Outputs: even
```

Else if Statement

```
int score = 99;
if (score == 100) {
    std::cout << "Superb";
}
else if (score >= 90) {
    std::cout << "Excellent";
}
else if (score >= 80) {
    std::cout << "Very Good";
}
else if (score >= 70) {
    std::cout << "Good";
}
else if (score >= 60)
    std::cout << "OK";
else
    std::cout << "What?";
```

Ternary Operator

```
Result = Condition ? Exp1 : Exp2;
```

Switch Statement

```
int num = 2;
switch (num) {
```

Operators

Relational Operators

`a == b` a is equal to b

`a != b` a is NOT equal to b

`a < b` a is less than b

`a > b` a is greater b

`a <= b` a is less than or equal to b

`a >= b` a is greater or equal to b

Assignment Operators

`a += b` Aka `a = a + b`

`a -= b` Aka `a = a - b`

`a *= b` Aka `a = a * b`

`a /= b` Aka `a = a / b`

`a %= b` Aka `a = a % b`

Logical Operators

```
// Outputs: 5
std::cout << max << std::endl;
```

```
int x = 3, y = 5, max;
if (x > y) {
    max = x;
} else {
    max = y;
}
// Outputs: 5
std::cout << max << std::endl;
```

```
std::cout << "One";
break;
case 2:
    std::cout << "Two";
    break;
case 3:
    std::cout << "Three";
    break;
default:
    std::cout << "What?";
    break;
}
```

!exp	exp is false (NOT)
Bitwise Operators	
a & b	Binary AND
a b	Binary OR
a ^ b	Binary XOR
~ a	Binary One's Complement
a << b	Binary Shift Left
a >> b	Binary Shift Right

C++ Loops

While

```
int i = 0;
while (i < 6) {
    std::cout << i++;
}

// Outputs: 012345
```

Do-while

```
int i = 1;
do {
    std::cout << i++;
} while (i <= 5);

// Outputs: 12345
```

Continue statements

```
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue;
    }
    std::cout << i;
} // Outputs: 13579
```

```

}

for (;;) {
    std::cout << "infinite loop";
}

for(int i = 1; i > 0; i++) {
    std::cout << "infinite loop";
}

```

```

int main()
{
    auto print = [](int num) { st

    std::array<int, 4> arr = {1,
    std::for_each(arr.begin(), ar
    return 0;
}

```

```

}
// Outputs: 1 2 3 4 5

std::string hello = "QuickRef.ME"
for (char c: hello)
{
    std::cout << c << " ";
}
// Outputs: Q u i c k R e f . M E

```

Break statements

```

int password, times = 0;
while (password != 1234) {
    if (times++ >= 3) {
        std::cout << "Locked!\n";
        break;
    }
    std::cout << "Password: ";
    std::cin >> password; // input
}

```

Several variations

```

for (int i = 0, j = 2; i < 3; i++)
    std::cout << "i=" << i << ","
    std::cout << "j=" << j << ";
}
// Outputs: i=0,j=2;i=1,j=1;i=2,j

```

C++ Functions


```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    std::cout << add(10, 20);  
}
```

add is a function taking 2 ints and returning int

```
}  
void fun(string a) {  
    std::cout << a;  
}  
void fun(int a) {  
    std::cout << a;  
}
```

```
int main() {  
    // sqrt() is from cmath  
    std::cout << sqrt(9);  
}
```

C++ Classes & Objects

Defining a Class

```
class MyClass {  
    public:                // Access specifier  
        int myNum;        // Attribute (int variable)  
        string myString;  // Attribute (string variable)  
};
```

Creating an Object

```
MyClass myObj; // Create an object of MyClass  
  
myObj.myNum = 15; // Set the value of myNum  
myObj.myString = "Hello"; // Set the value of myStr:  
  
cout << myObj.myNum << endl; // Output 15  
cout << myObj.myString << endl; // Output "Hello"
```

```

string myString;
MyClass() { // Constructor
    myNum = 0;
    myString = "";
}
};

MyClass myObj; // Create an object of MyClass

cout << myObj.myNum << endl;           // Output 0
cout << myObj.myString << endl;        // Output ""

```

```

string myString;
MyClass() { // Constructor
    myNum = 0;
    myString = "";
}
~MyClass() { // Destructor
    cout << "Object destroyed." << endl;
}
};

MyClass myObj; // Create an object of MyClass

// Code here...

// Object is destroyed automatically when the program ends

```

Class Methods

```

class MyClass {
public:
    int myNum;
    string myString;
    void myMethod() { // Method/function defined inside the class
        cout << "Hello World!" << endl;
    }
}

```

Access Modifiers

```

class MyClass {
public:           // Public access specifier
    int x;       // Public attribute
private:        // Private access specifier
    int y;       // Private attribute
protected:     // Protected access specifier
    int z;       // Protected attribute
}

```

```
myObj.y = 50; // Not allowed (private)
myObj.z = 75; // Not allowed (protected)
```

Getters and Setters

```
class MyClass {
private:
    int myNum;
public:
    void setMyNum(int num) { // Setter
        myNum = num;
    }
    int getMyNum() { // Getter
        return myNum;
    }
};

MyClass myObj;
myObj.setMyNum(15); // Set the value of myNum to 15
cout << myObj.getMyNum() << endl; // Output 15
```

Inheritance

```
class Vehicle {
public:
    string brand = "Ford";
    void honk() {
        cout << "Tuut, tuut!" << endl;
    }
};

class Car : public Vehicle {
public:
    string model = "Mustang";
};

Car myCar;
myCar.honk(); // Output "Tuut, tuut!"
cout << myCar.brand + " " + myCar.model << endl; //
```

C++ Preprocessor

else	endif
------	-------

ifdef	ifndef
-------	--------

define	undef
--------	-------

include	line
---------	------

error	pragma
-------	--------

defined	__has_include
---------	---------------

__has_cpp_attribute	export
---------------------	--------

import	module
--------	--------

#undef F00

```
#if VERSION == 2.0
    #error Unsupported
    #warning Not really supported
#endif
```

#define DEG(x) ((x) * 57.29)

```
#define DST(name) name##_s name##
DST(object);    #=> object_s objec
```

```
#ifdef DEBUG
    console.log('hi');
#elif defined VERBOSE
    ...
#else
    ...
#endif
```

```
#define STR(name) #name
char * a = STR(object);    #=> cha
```

```
#define LOG(msg) console.log(__FI
#=> console.log("file.txt", 3, "r
```

Miscellaneous

\b

Backspace

Escape Sequences

Keywords

\r	Return
\t	Horizontal tab
\v	Vertical tab
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark
\?	Question mark
\0	Null Character

Preprocessor	
if	elif
else	endif
ifdef	ifndef
define	undef
include	line
error	pragma

bitor	bool	break	case	catch
char	char8_t	char16_t	char32_t	class
compl	concept	const	constexpr	constexpr
constinit	const_cast	continue	co_await	co_return
co_yield	decltype	default	delete	do
double	dynamic_cast	else	enum	explicit
export	extern	false	float	for
friend	goto	if	inline	int
long	mutable	namespace	new	noexcept
not	not_eq	nullptr	operator	or
or_eq	private	protected	public	constexpr
register	reinterpret_cast	requires	return	short
signed	sizeof	static	static_assert	static_cast
struct	switch	synchronized	template	this
thread_local	throw	true	try	typedef

[import](#)[module](#)[xor](#)[xor_eq](#)[final](#)[override](#)[transaction_safe](#)

Also see

[C++ Infographics & Cheat Sheets](#)

([hackingcpp.com](#))

[C++ reference](#) ([cppreference.com](#))

[C++ Language Tutorials](#) ([cplusplus.com](#))

Top Cheatsheet

Python Cheatsh
Quick Reference

Vim Cheatsheet
Quick Reference

InfrontJS Cheat
Quick Reference

Remote Work R
Quick Reference

JavaScript Chea
Quick Reference

Bash Cheatshee
Quick Reference

Homebrew Chea
Quick Reference

PyTorch Cheats
Quick Reference

Recent Cheatsheet

QuickRef.ME

Share quick reference and cheat sheet for developers.

#Notes



© 2026 QuickRef.ME, All rights reserved.